

Lección 8. Control de flujos de un programa



Control de flujos de un programa

INICIAR >



Control de flujos de un programa

Introducción

La **creación de hilos** dentro de los procesos permite separar las tareas que se van a efectuar; de este modo, se pueden ejecutar al mismo tiempo.

Lenguajes como *Java* ofrecen librerías que abstraen la complejidad de los procesos para la implementación de hilos, de una forma sencilla y rápida.



Control de flujos de un programa

Concepto de hilo

Un **proceso** está compuesto de hilos (de uno a varios), los cuales también son llamados **procesos ligeros**.

Los hilos son secuencias de subtareas de un proceso.

Estos usan la memoria del proceso ya que no cuentan con una propia para poder ejecutarse.







Control de flujos de un programa

Concepto de hilo

Algunas **ventajas de usar hilos** son:

- En aplicaciones grandes con tareas múltiples, aceleran la ejecución y el tiempo de respuesta del programa.
- Mejoran el rendimiento de aplicaciones complejas.

Recuerda:

Los procesos son programas en ejecución, los cuales cuentan con un estado propio.

Los hilos también se denominan como procesos ligeros.





Control de flujos de un programa

Operaciones básicas de los hilos

La implementación de hilos en tus programas es más sencilla y rápida con *Java*.

Existen dos formas de crear hilos:

- Heredando la clase Thread o implementando la interface Runnable
- Para después, pasarlo como parámetro a una instancia de la clase Thread

Cualquiera de estas opciones ofrece una serie de métodos que puedes usar para controlar los hilos que ejecutes.

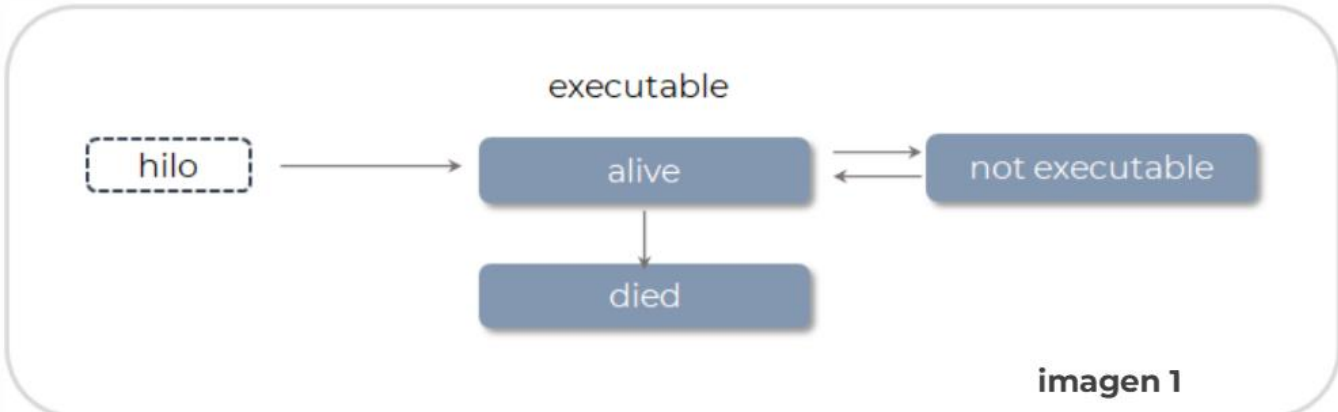


Control de flujos de un programa

Operaciones básicas de los hilos

Observa detenidamente la siguiente explicación de la imagen 1 + Pulsa para ver

executable



hilo

alive

died

not executable

imagen 1



Control de flujos de un programa

Operaciones básicas de los hilos

Observa detenidamente la siguiente explicación de la imagen 1

- Al iniciarse, un hilo entra en estado **alive** (vivo), que mediante métodos como `sleep()` o `wait()`, pasa al estado **no ejecutable**.
- El estado **no ejecutable** detiene temporalmente la ejecución del hilo.
- Una vez que el hilo ha terminado de ejecutar el método `run()`, éste pasa al estado **died** (muerto/terminado).

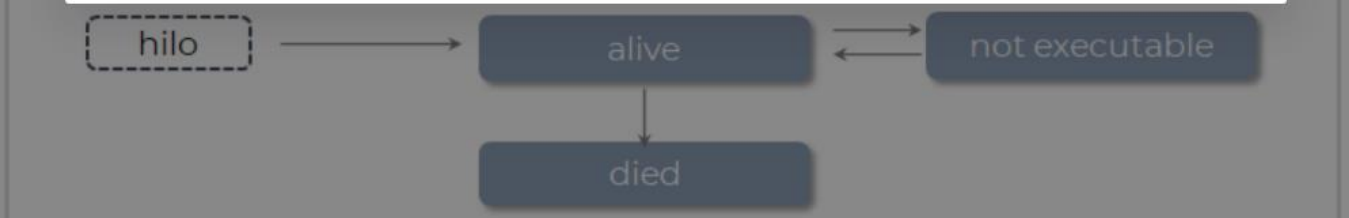


imagen 1



Control de flujos de un programa

Operaciones básicas de los hilos

A continuación, te invito a revisar las operaciones básicas de los hilos.
Pulsa sobre cada botón.

notifyAll()

notify()

setPriority()

start()

run()

yield()

interrupt()

isAlive()

sleep(time)

wait()

Operaciones básicas

Operaciones básicas:

start()

Este método se encarga de correr el hilo, se invoca después de su instanciación.

Al iniciar con el **método** *start()*, un hilo entra en estado vivo, donde pasa al estado ejecutable o al no ejecutable.

run()

Cuando creas un hilo con el método *run()*, su código se sobrescribe con el código del subproceso que va a ejecutar.

Para iniciar el hilo se debe invocar el método *start()*, nunca el propio *run()*.

isAlive()

Este método regresa un valor booleano (*true* o *false*) que indica si el hilo se encuentra en estado vivo.

sleep(time)

Detiene el hilo que está en ejecución durante el tiempo que se encuentre como parámetro (este tiempo es especificado en milisegundos).

El método *sleep* te permite dormir al hilo mismo que volverá al estado no ejecutable.

wait()

Detiene el hilo que está en ejecución hasta que un método *notify()* o *notifyAll()* sea invocado.

interrupt()

Este método se encarga de detener el hilo que se encuentra en ejecución.

yield()

El método *yield* se invoca cuando se quiere sugerir que el hilo ceda su lugar a los otros hilos.

El procesador no siempre hace caso de este método, por lo que solo es recomendado para hacer pruebas o depurar del código.

setPriority()

Todos los hilos que son ejecutados cuentan con una prioridad, la cual es tomada en cuenta para ejecutar un hilo antes que otro dentro de las secciones críticas.

La prioridad puede ser reasignada con el método *setPriority*. Puede asignar la prioridad de cada hilo mediante dos valores aceptados:

MIN_PRIORITY y *MAX_PRIORITY*.

notify()

El método *notify* notifica a un hilo específico que puede seguir con su ejecución, después de que un método *wait()* lo había detenido.

notifyAll()

Notifica a todos lo hilos cuya ejecución ha sido detenida por el método *wait()*, que pueden continuar con su tarea.

The infographic is titled "Control de flujos de un programa" and "Tipos de flujos". It features a background image of a blue box with the "uveg" logo (Universidad Virtual del Estado de Guerrero) and a circuit board. The main content is enclosed in a white box with a grey border. At the top, it states: "Existen **dos tipos de flujo** dentro de los programas que codificas:". Below this, there is a list: "• Flujo único" and "• Flujo múltiple". A curved arrow points from the "Flujo único" item to a yellow box on the left. The yellow box contains the text: "Hasta el momento, la manera en que has programado es de forma secuencial, en la que cada tarea, para ser ejecutada, debe esperar a que termine la anterior." and "Esta técnica se conoce como **programación en flujo único.**". Another curved arrow points from the "Flujo múltiple" item to a blue box on the right. The blue box contains the text: "**El flujo único,** también conocido como ***single thread.***" and "Usa un hilo principal mismo que es necesario declarar o implementar de manera alguna." On the right side of the infographic, there are three circular icons: a lock icon at the top, a home icon in the middle, and a more options icon at the bottom.

Control de flujos de un programa

Tipos de flujos

Existen **dos tipos de flujo** dentro de los programas que codificas:

- Flujo único
- Flujo múltiple

Hasta el momento, la manera en que has programado es de forma secuencial, en la que cada tarea, para ser ejecutada, debe esperar a que termine la anterior.

Esta técnica se conoce como **programación en flujo único.**

El flujo único, también conocido como ***single thread.***

Usa un hilo principal mismo que es necesario declarar o implementar de manera alguna.



Control de flujos de un programa

Tipos de flujos



Observa el siguiente ejemplo

En la **imagen 2** puedes observar un ejemplo sencillo elaborado en *Java* que seguro verás con bastante familiaridad, es un clásico

```
public class HolaMundo {  
    public static void main(String[] args) {  
        System.out.println("¡Hola mundo!");  
    }  
}
```

Imagen 2

¡Hola mundo!, que sin haberlo implementado explícitamente, ejecuta un **flujo único**.



Control de flujos de un programa

Tipos de flujos

El flujo múltiple, también conocido como **multitarea** o *multi-thread*

Implica el **uso de hilos múltiples** para:

- Implementar varias tareas que, aparentemente, se ejecutan al mismo tiempo, lo que brinda un mejor aprovechamiento de los tiempos de ejecución.

Este hilo usará un nombre para identificarse

Se redefine el método `run()` con el código que ejecutará el hilo

Se crean los hilos y se inicia su ejecución mediante el método `start()`

```
public class Hilo extends Thread {  
    String nombre;  
  
    public Hilo(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void run() {  
        try {  
            System.out.println(this.nombre + " duerme");  
            Thread.sleep(2000);  
            System.out.println(this.nombre + " ha despertado");  
        } catch (Exception e) {  
            System.out.println(e);  
        }  
        System.out.println(this.nombre + " ha sido ejecutado.");  
    }  
  
    public static void main(String[] args) {  
        Hilo t1 = new Hilo("Hilo 1");  
        Hilo t2 = new Hilo("Hilo 2");  
        Hilo t3 = new Hilo("Hilo 3");  
  
        t1.start();  
        t2.start();  
        t3.start();  
    }  
}
```

Ampliar

El código que implementas con **el método run()**:

1. Imprime un mensaje que avisa cuál hilo mandarás a **dormir**.
2. Con la función *sleep* de la clase *thread*, se detiene la ejecución del hilo por 2000 milisegundos.
3. Imprime un mensaje que avisa qué hilo ha terminado de **dormir**.
4. Imprime un mensaje que se ha terminado la ejecución.

El **método sleep** debe ser implementado dentro de una sentencia *try... catch* que lanza una excepción en caso de que ocurra un problema al ejecutar el código

```
public class Hilo extends Thread{
    String nombre;

    public Hilo(String nombre){
        this.nombre = nombre;
    }

    public void run(){
        try {
            System.out.println(this.nombre + " duerme"); 1
            Thread.sleep(2000);
            System.out.println(this.nombre + " ha despertado"); 3
        } catch (Exception e) {
            System.out.println(e);
        }
        System.out.println(this.nombre + " ha sido ejecutado."); 4
    }

    public static void main(String[] args) {
        Hilo t1 = new Hilo("Hilo 1");
        Hilo t2 = new Hilo("Hilo 2");
        Hilo t3 = new Hilo("Hilo 3");

        t1.start();
        t2.start();
        t3.start();
    }
}
```

Control de flujos de un programa

Tipos de flujos

Al ejecutar el código anterior puedes tener diferentes resultados, ya que cada hilo entra sin esperar a que el otro termine su proceso.

Ejecución 1

```
run:
Hilo 1 duerme
Hilo 3 duerme
Hilo 2 duerme
Hilo 3 ha despertado
Hilo 3 ha sido ejecutado.
Hilo 2 ha despertado
Hilo 1 ha despertado
Hilo 1 ha sido ejecutado.
Hilo 2 ha sido ejecutado.
BUILD SUCCESSFUL (total time: 2 seconds)
```

Ejecución 2

```
run:
Hilo 1 duerme
Hilo 3 duerme
Hilo 2 duerme
Hilo 2 ha despertado
Hilo 2 ha sido ejecutado.
Hilo 3 ha despertado
Hilo 3 ha sido ejecutado.
Hilo 1 ha despertado
Hilo 1 ha sido ejecutado.
BUILD SUCCESSFUL (total time: 2 seconds)
```

Ejecución 3

```
run:
Hilo 1 duerme
Hilo 3 duerme
Hilo 2 duerme
Hilo 1 ha despertado
Hilo 3 ha despertado
Hilo 3 ha sido ejecutado.
Hilo 2 ha despertado
Hilo 2 ha sido ejecutado.
Hilo 1 ha sido ejecutado.
BUILD SUCCESSFUL (total time: 2 seconds)
```




Control de flujos de un programa

Flujo único vs. Flujo múltiple



Ventajas

- En múltiples ocasiones, los programas que escribes son de complejidad básica.
- Por lo cual las ventajas que puede ofrecerte el **flujo múltiple** (como la ejecución de más de una tarea al mismo tiempo) no lo son en todos los casos.

- El **flujo único** es más sencillo de programar.



- Lenguajes como *Java* ofrecen la facilidad de crear hilos múltiples para controlar los hilos y ejecutar diferentes tareas al mismo tiempo.



Control de flujos de un programa

Flujo único vs. Flujo múltiple



Desventajas

- Cuando se usa el **flujo único** en programas con una complejidad mayor, los tiempos de ejecución se alargan.

- Ejecución secuencial de cada tarea programada en el hilo principal.



- Cuando quieras programar con hilos, es necesario que planees dónde los implementarás para que los aproveches al máximo.
- Esto te implicará un esfuerzo extra en relación con el que se emplea en el flujo único.





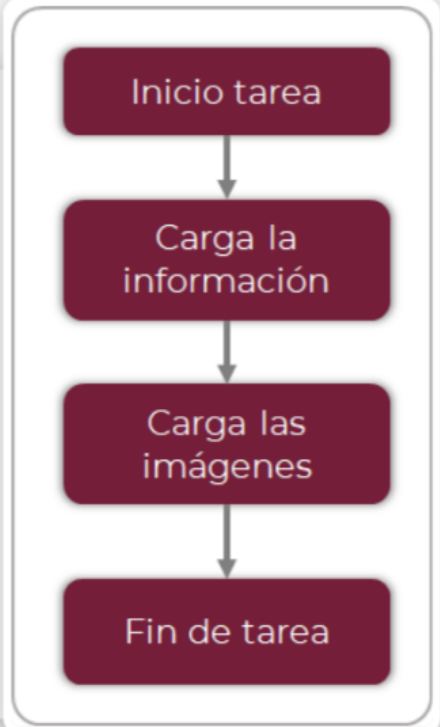
Control de flujos de un programa



Observa detenidamente un ejemplo que te presenta lo estudiado

Si **al cargar una página web** inicias esta tarea cargando primero el texto y después las imágenes, se notará al inicio si la conexión no es muy rápida.

Flujo único vs. Flujo múltiple



```
graph TD; A[Inicio tarea] --> B[Carga la información]; B --> C[Carga las imágenes]; C --> D[Fin de tarea];
```



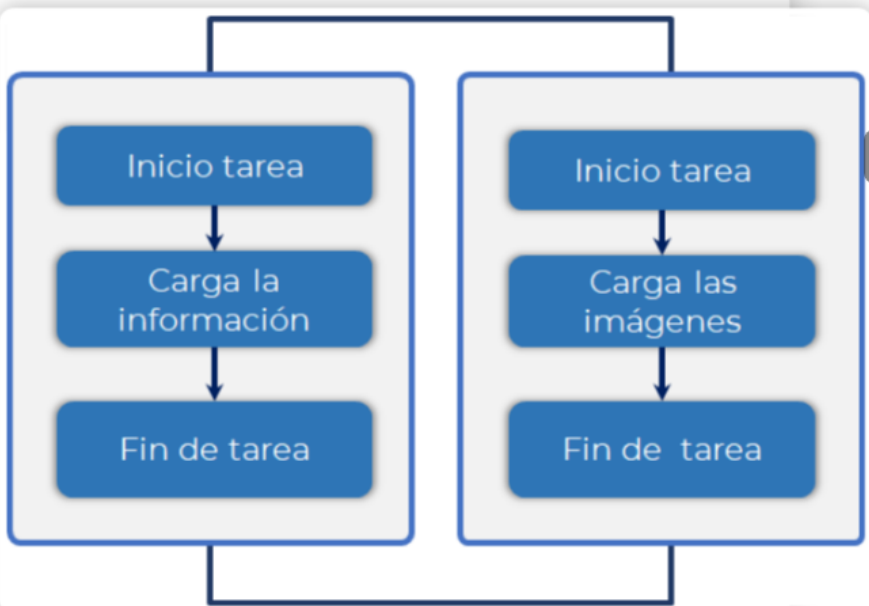
Control de flujos de un programa



Observa detenidamente un ejemplo que te presenta lo estudiado

Si **al cargar una página web** inicias con dos tareas, cada una encargada de realizar sus acciones independientes una de la otra, podrás ver cómo se aprovecha la memoria de los procesos en ambas tareas.

Flujo único vs. Flujo múltiple



```
graph TD; subgraph Task1 [ ]; A1[Inicio tarea] --> B1[Carga la información]; B1 --> C1[Fin de tarea]; end; subgraph Task2 [ ]; A2[Inicio tarea] --> B2[Carga las imágenes]; B2 --> C2[Fin de tarea]; end;
```



Control de flujos de un programa

Control de flujos de un programa



IMPORTANTE

Java te ofrece

Una forma de incluir hilos en tus programas de una manera sencilla, lo que permite mejorar el rendimiento de tus proyectos.



La implementación de hilos en diferentes tareas agiliza los trabajos y ofrece mejores soluciones, lo que proporciona al cliente un proyecto más acorde a sus necesidades.

Recuerda que actualmente el usuario final busca velocidad y eficiencia.

